HTTP Handler Decomposition

Route Registrar Extraction

Epic status: Implemented

Work orders: WO-084 · WO-085 · WO-091–096 · WO-102 · WO-109

Default behavior: Unchanged (legacy path)

Agenda

1. Problem & goals
2. Architecture (Strangler Fig)
3. Work-order delivery map
4. Safety nets (allowlist + parity CI)
5. Testing & verification
6. Risks, caveats & next steps

The problem

Vault's HTTP API routing lived in a **monolithic** `handlerWithSettings` in `http/handler.go`.

- ~150 lines of inline `mux.Handle(...)` registrations
- Hard to review ownership per subsystem
- Easy for unauthenticated routes to drift unnoticed
- Blocked safe, incremental extraction

Goals

Goal	Outcome
Declarative unauth allowlist	CI fails on divergence
Shared route context	Namespace + chroot helpers
Named registrars	Recovery, system, auth, secrets, identity, UI, telemetry
Feature-flag gate	Decomposed vs legacy
Parity automation	Legacy ↔ decomposed route inventory diff

Pattern: Strangler Fig



Today: flag off → legacy → **zero user-visible change**

Architecture after change

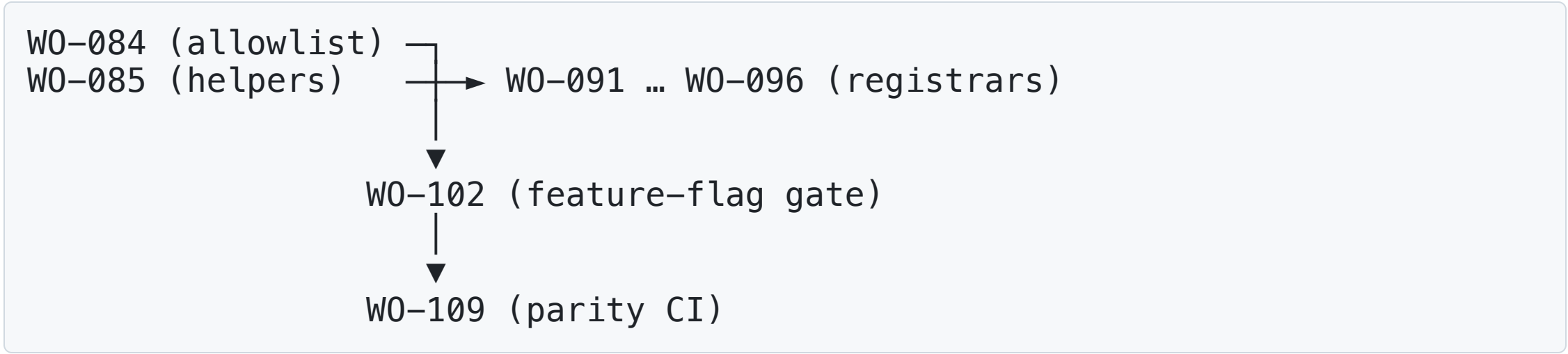
Shared context (`http/handler_helpers.go`)

- `RouteContext` — Core, Mux, OperatorNamespace, ChrootNamespace, Settings, Props
- `NewRouteContext` — extracts chroot + operator namespace once
- `SysPath(suffix)` — `/v1/{ns}sys/{suffix}`

Orchestration (`http/handler.go`)

- Recovery always uses `registerRecoveryRoutes`
- Non-recovery gated by `IsFeatureFlagEnabled("http-handler-decomposition")`

Dependency order



Start helpers + allowlist in parallel → extract registrars → gate → inventory check.

WO-084 — Unauthenticated allowlist

Deliverables

- `http/unauthenticated_routes.go` — `UnauthenticatedRoutes`
- `http/unauthenticated_routes_test.go` — AST parse of Handle registrations
- `.github/workflows/route-allowlist.yml`

Acceptance

- Declarative Go data structure
- CI rejects allowlist / registration drift
- Read-only metadata — no runtime behavior change

WO-085 — RouteContext helpers

Deliverables

- `http/handler_helpers.go`
- `http/handler_helpers_test.go`

Covers

- ChrootNamespace from ListenerConfig
- `SysPath("init")` → `/v1/sys/init`
- Operator namespace → `/v1/admin/sys/init`
- Nested namespaces

`handlerWithSettings` uses `NewRouteContext` (no duplicated prefix logic).

WO-091 — Recovery routes

File: `http/handler_recovery.go` → `registerRecoveryRoutes`

Exactly **3** routes:

1. `/v1/sys/raw/`
2. `/v1/sys/generate-recovery-token/attempt`
3. `/v1/sys/generate-recovery-token/update`

Test: `handler_recovery_test.go`

(`RecoveryToken` is `*atomic.String`)

WO-092 — System routes

File: `http/handler_system.go` → `registerSystemRoutes`

- Unauthenticated sys paths (init, seal, unseal, health, …)
- Conditional generate-root / rekey (settings flags)
- Raft bootstrap/join + feature-flags
- Inject-data top routes
- Catch-alls: `/v1/sys/` and `/v1/`

WO-093 & WO-094 — Auth · Secrets · Identity

Registrar	File	Behavior
Auth	`handler_auth.go`	DR op token routes when `unauthDROperationToken`
Secrets	`handler_secrets.go`	**No-op** extension point
Identity	`handler_identity.go`	**No-op** extension point

Secrets/identity still flow through the catch-all `handleLogical` / radix router until further extraction.

WO-095 & WO-096 — UI · Telemetry

UI (`handler_ui.go`)

- Early return if `!UIEnabled()`
- Built-in assets vs stub
- `/ui/`, `/robots.txt`, `/ui`, `/`

Telemetry (`handler_telemetry.go`)

- Metrics, pprof, in-flight-req
- Conditional unauthenticated access from ListenerConfig

WO-102 — Feature-flag gate

```
if rc.Core.IsFeatureFlagEnabled("http-handler-decomposition") {
    // Decomposed path — Strangler Fig
    registerSystemRoutes(rc)
    registerAuthRoutes(rc)
    registerSecretsRoutes(rc)
    registerIdentityRoutes(rc)
    registerUIRoutes(rc)
    registerTelemetryRoutes(rc)
} else {
    registerLegacyRoutes(rc)  // verbatim fallback
}
```

- Legacy body preserved in `http/handler_legacy.go`
- OSS stub: `IsFeatureFlagEnabled` → `IsFlagEnabled` (**always false**)

WO-109 — Route inventory parity

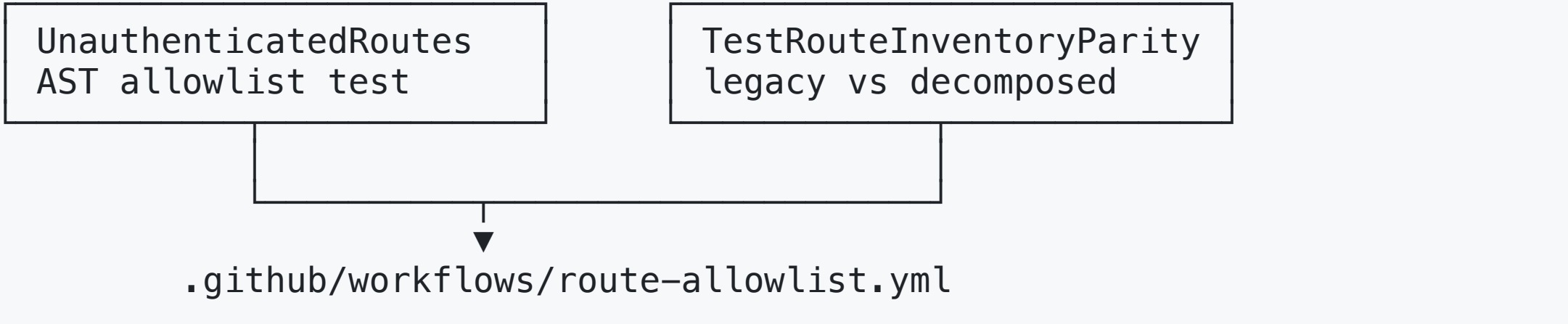
File: `http/route_inventory_test.go`

1. Register legacy routes → extract patterns
2. Register decomposed routes → extract patterns
3. `cmp.Diff` — fail on any divergence

Also run from `route-allowlist.yml` on PR/push.

Helper: `extractRegisteredRoutes` probes candidate paths on `ServeMux` (Go 1.22+ has no public pattern list).

Safety nets



Prevents silent unauth surface growth **and** registrar drift.

Testing

```
GOWORK=off go test ./http/ -run \
  'TestUnauthenticatedRouteAllowlist|TestRouteInventoryParity|TestRegister|TestRouteContext|TestNewRouteContext|TestHandlerWithSettings' \
  -count=1
```

Why `GOWORK=off`?

Workspace includes `tools/pipeline` → private `github.com/hashicorp/releases-api`. App tests don't need it.

Result: Targeted suite executed successfully after RecoveryToken type fix.

Files inventory

New: allowlist, helpers, recovery/system/auth/secrets/identity/ui/telemetry/legacy registrars + tests, route inventory test, CI workflow

Modified:

- `http/handler.go` — orchestration + gate
- `vault/core_stubs_oss.go` — `IsFeatureFlagEnabled` stub

Caveats & follow-ups

Item	Status
WO-083 / WO-090 (real feature flags)	Stub only on OSS; flag stays off
WO-119 (byte-identical responses)	Out of this epic
Secrets / identity extraction	Registrars are placeholders
Production cutover	Enable flag only after soak + parity stays green

Talking points (elevator)

1. **Why** — Monolith blocked ownership and safe change
2. **What** — Named registrars + allowlist + inventory CI
3. **How** — Strangler Fig behind `http-handler-decomposition`
4. **Default** — Legacy path; no production behavior change
5. **Proof** — AST allowlist + `cmp.Diff` parity + registrar unit tests
6. **Next** — Real flag wiring, optional WO-119, gradual enablement

Summary

> We extracted Vault's HTTP route registration into subsystem registrars with allowlist and inventory CI, while keeping the legacy path as the default behind a feature flag so production behavior stays unchanged until an intentional cutover.

Questions?